LAPORAN PRAKTIKUM MICROPROCESSOR & INTERFACE

PENJUMLAHAN DATA HEXADECIMAL



PRAKTIKAN

Nama : Elsadanika Maura Sitorus

NIM : 211344013

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022/2023

KETERANGAN

1. Kelompok : -
2. Judul Laporan : Penjumlahan Data Hexadecimal
3. Tanggal Praktikum : 24 Januari 2023
4. Tanggal Pengumpulan : 28 Januari 2023
5. Nama Praktikan : Elsadanika Maura Sitorus
6. Nama Partner : -
7. Nama Dosen Pembimbing : Ferry Satria, BSEE., M. T.
   Rahmawati Hasanah, S. ST., M. T.

DAFTAR ISI

KETERANGAN i
DAFTAR ISI ii

A. Tujuan Praktikum 1
B. Dasar Teori 1
C. Alat dan Komponen yang Digunakan 3
D. Langkah Kerja 4
E. Data Pengamatan 5
F. Analisis Data 9
G. Kesimpulan 12
H. Daftar Pustaka 13

A. Tujuan Praktikum

1. Memahami bahasa Assembly
2. Memahami cara penggunaan simulator EMU 8086
3. Memahami pemanfaatan EMU 8086 simulator untuk menjumlahkan data dalam bentuk hexadecimal.

B. Dasar Teori

1. Definisi Mikroprosesor

Mikroprosesor merupakan CPU (Central Processing Unit) yang dikemas dalam satu buah IC (Integrated Circuit) atau chip. Mikroprosesor memiliki kemampuan untuk menerima input data digital kemudian diproses sesuai intruksi dalam memori untuk menghasilkan output. Mikroprosesor beroperasi dalam sistem angka biner dengan logika kombinasional dan logika digital sekuensial. Didalamnya terdapat ALU (unit aritmatika dan logika), register, control unit, bus, dan clock untuk komputasi.

Mikroprosesor mampu menjalankan perintah secara berurutan dalam bentuk program dengan intruksi yang menggunakan sistem bilangan biner. Akan tetapi, tidak semua mikroprosesor kompatibel untuk menjalankan berbagai program dalam kode intruksi yang berbeda. Beberapa jenis mikroprosesor seperti Z80 hanya dapat menjalankan intruksi untuk mikroprosesor 8080 atau 8085, sedangkan program yang menggunakan prosesor 8086 hanya dapat dijalankan dengan prosesor yang lebih tinggi seperti 80186, 80286, dan seterusnya. Sebaliknya, program 80186, 80286 dan seterusnya tidak berkemampuan untuk dijalankan dengan prosesor dibawahnya.

2. Sejarah Mikroprosesor

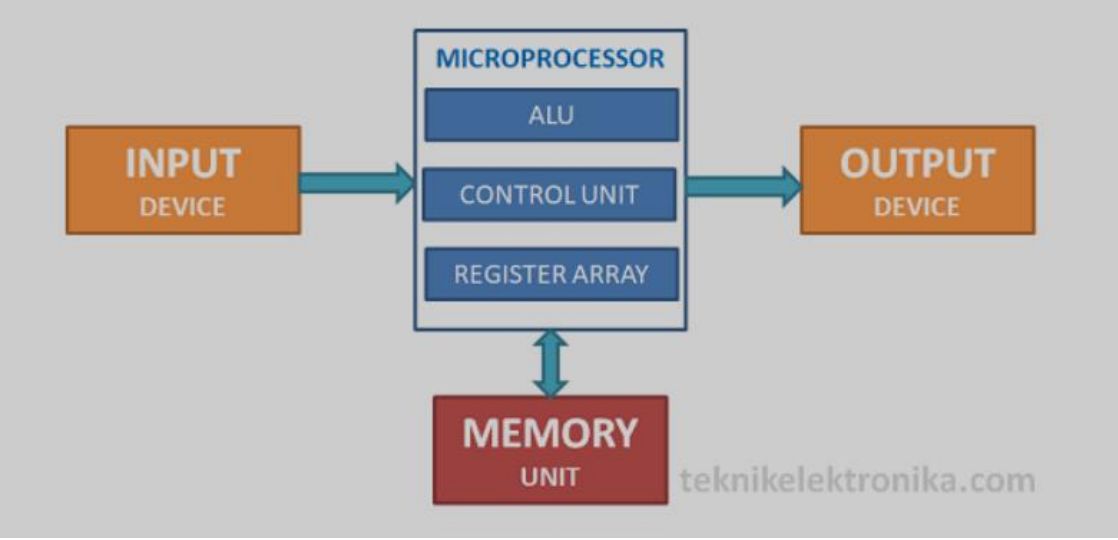
Mikroprosesor bermula dari ditemukannya IC oleh Fair Child Semikonduktor di tahun 1959. Pada tahun 1968, ahli kimia Gordan Moore, fisikawan Robert Noyce, dan investor Andrew Grove yang merupakan bagian dari Fair Child Semikonduktor mengundurkan diri dan mulai mendirikan perusahaan Integrated Electronics (INTEL). Semakin berkembang, pada tahun 1971 INTEL menciptakan mikroprosesor pertama dengan nama Intel 4004 dengan arsitektur 4 bit.

Dalam upaya pengembangannya, Intel 4004 diberi penambahan peripheral berupa memori, piranti I/O, dan sebagainya kemudian diubah menjadi sebuah computer kecil. Mikroprosesor Intel 4004 memiliki 2.250 transistor PMOS, dapat mengeksekusi 60 ribu operasi tiap detik, dan menangani data 4 bit. Mikroprosesor dikembangkan lagi menjadi 8080 dengan 8 bit arsitektur dengan 4.500 transistor dengan kinerja 10 kali dari seri 4004, dilanjutkan dengan 8085, lalu 8086 dengan arsitektur 16 bit.

3. Fungsi dan Keunggulan Mikroprosesor

Fungsi utama sebuah mikroprosesor adalah sebagai unit pengendali kerja sistem mikroprosesor. Fungsi lainnya adalah mengambil data dan intruksi dari memori, memindahkan data dari dan ke memori, mengirim sinyal kendali dan menerima sinyal interupsi, melaksanakan fungsi operasi logika dan aritmatika, serta melakukan timing untuk siklus kerja sistem mikroprosesor (Hidayatullah, 2021). Sedangkan keunggulan dari mikroprosesor adalah memiliki kecepatan tinggi dalam menjalankan intruksi, portable karena ukurannya yang kecil dengan konsumsi daya rendah, multiguna, biaya rendah, dan tidak menghasilkan panas berlebih.

4. Cara Kerja Mikroprosesor



Gambar 1 Diagram Blok Mikroprosesor pada Komputer

Mikroprosesor menjalan intruksi berdasarkan penjemputan dan pengambilan perintah dan data yang diperlukan (Fetch), pembacaan sandi (Decode), dan menjalankan perintah (Execute). Intruksi akan disimpah dalam unit memori secara berurutan, lalu mikroprosesor mengambil intruksi tersebut dari memory kemudian

menerjemahkan serta mengeksekusi intruksi hingga mendapat intruksi berhenti. Hasil eksekusi akan dikirimkan dalam bentuk biner ke output. Dalam proses ini, register array memiliki fungsi untuk menyimpan data yang bersifat sementara, dan ALU untuk melakukan fungsi komputasi.

5. Istilah-Istilah dalam Mikroprosesor

Berikut ini merupakan istilah umum dalam mikroprosesor:

- BUS

  BUS yaitu seperangkat konduktor untuk mengirim data, alamat, atau mengontrol informasi ke elemen-elemen pada mikroprosesor. Terdapat 3 jenis BUS, BUS Data, BUS Kontrol, BUS Alamat.

- Instruction Set

  Intruction Set merupakan perintah yang dapat dipahami oleh mikroprosesor. Intruksi memerintahkan prosesor untuk mengganti transistor yang relevan melakukan pemrosesan data.

- Word Length

  Word Length merupakan jumlah bit dalam BUS data prosesor.

- Cache Memory

  Memori cache merupakan memori akses yang acak dan terintegrasi dalam prosessor sehingga prosesor dapat mengakses data dalam memori cache dengan lebih cepat daripada RAM biasa.

- Clock Speed

  Sinyal clock yang digunakan untuk mengontrol laju intruksi yang, menyinkronisasi komponen internal mikroprosesor, dan mengendalikan transfer data. Clock speed mengacu pada kecepatan mikroprosesor dalam menjalankan intruksi yang diukur dalam Hertz.

6. Perintah Looping, Memory, dan Data Area.

Arahan assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte.

a) Mengalokasikan Ruang Penyimpanan untuk Data yang Diinisialisasi

Sintaks untuk pernyataan alokasi penyimpanan untuk data yang diinisialisasi adalah sebagai berikut:

```
[nama-variabel] define-directive initial-value [,initial-value]...
```

Nama variabel adalah pengidentifikasi untuk setiap ruang penyimpanan. Assembler mengaitkan nilai offset untuk setiap nama variabel yang ditentukan dalam segmen data. Ada lima bentuk dasar dari direktif define yang ditampilkan pada tabel berikut:

Perintah	Keterangan	Storage Space
DB	Define Byte	1 byte
DW	Define Word	2 bytes
DD	Define Doubleword	4 bytes
DQ	Define Quadword	8 bytes
DT	Define Ten Bytes	10 bytes

b) Perintah Looping

Instruksi untuk transfer perintah selama eksekusi disesuaikan dengan kondisinya, berikut ini adalah instruksi untuk proses looping dalam Bahasa assembly pada 8086:

Instructions	Conditions
JA/JNBE	Jump if above/not below/equal instruction satisfies
JAE/JNB	Jump if above/not below
JBE/JNA	Jump if below/equal/ not above
JC	Jump if carry flag CF = 1
JE/JZ	Jump if equal/zero flag ZF = 1
JG/JNLE	Jump if greater/not less than/equal
JGE/JNL	Jump if greater than/equal/not less
JL/JNGE	Jump if less than/not greater than/equal
JLE/JNG	Jump if less than/equal/if not greater
JNC	Jump if no carry flag (CF = 0)
JNE/JNZ	Jump if not equal/zero flag ZF = 0
JNO	Jump if no overflow flag OF = 0
JNP/JPO	Jump if not parity/parity odd PF = 0
JNS	Jump if not sign SF = 0
JO	Jump if overflow flag OF = 1
JP/JPE	Jump if parity/parity even PF = 1
JS	Jump if sign flag SF = 1

C. Alat dan Komponen yang Digunakan

- Laptop/Personal Computer
- Simulator EMU 8086

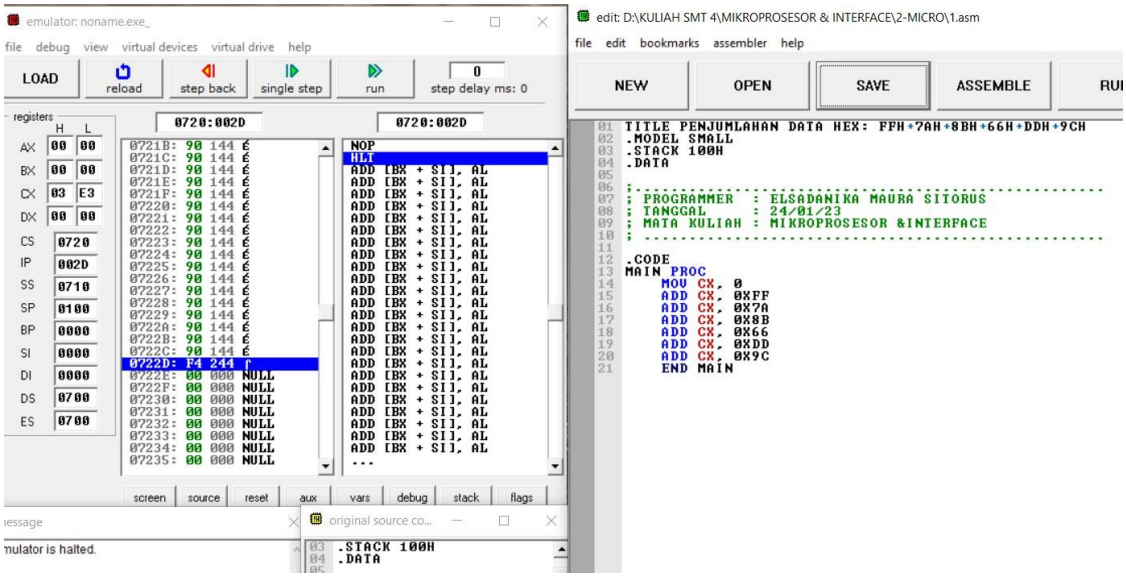
D. Langkah Kerja

1. Buatlah program untuk menjumlahkan 6 data heksadesimal berikut: FFH, 7AH, 8BH, 66H, DDH, 9CH. Simpan hasil penjumlahan pada register: CX.
2. Buatlah program untuk menjumlahkan 10 data heksadesimal berikut: FFH, 7AH, 8BH, 66H, DDH, 9CH, 7FH, 77H, 5CH, dan EEH. Simpan hasil penjumlahan pada register: CX.
3. Buatlah program untuk menjumlahkan 6 data heksadesimal berikut: FFFFH, DD7AH, 448BH, AA66H, 88DDH, 9CFFH. Simpan hasil penjumlahan pada register: BL:CX.
4. Buatlah program untuk menjumlahkan 6 data heksadesimal berikut: FFFFFFH, 7AAF45H, 8B6543H, FFFF66H, DDE4C9H, 9CDDEEH. Simpan hasil penjumlahan pada register: CX:DX.
5. Buatlah program untuk menjumlahkan 20 data heksadesimal berikut: FFH, 7AH, 8BH, 66H, DDH, 9CH, 88H, 54H, B7H, 99H, 7EH, 9DH, 5EH, 2BH, 66H, 4FH, C5H, 7AH, AAH, 9CH. Simpan hasil penjumlahan pada register CX.
6. Buat program untuk menjumlahkan 6 data heksadesimal yang berada pada register AX, BX, CX, DX, BP, dan DI. Simpan hasil penjumlahan pada Data Area dengan nama variabel “HASIL”.

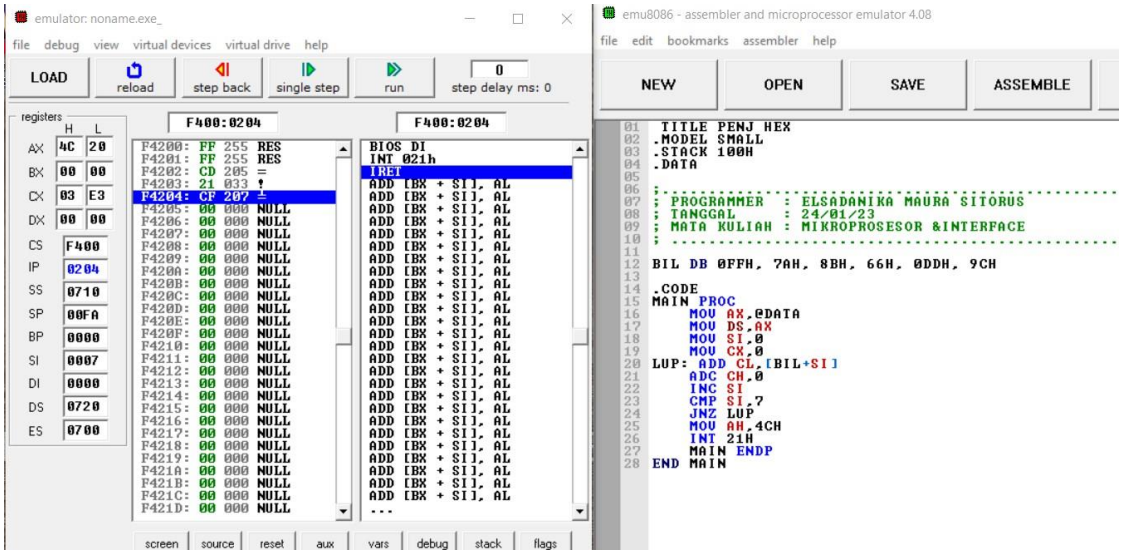
E. Data Pengamatan

1. FFH, 7AH, 8BH, 66H, DDH, 9CH. Simpan di CX.

   a) Perintah 16 bit

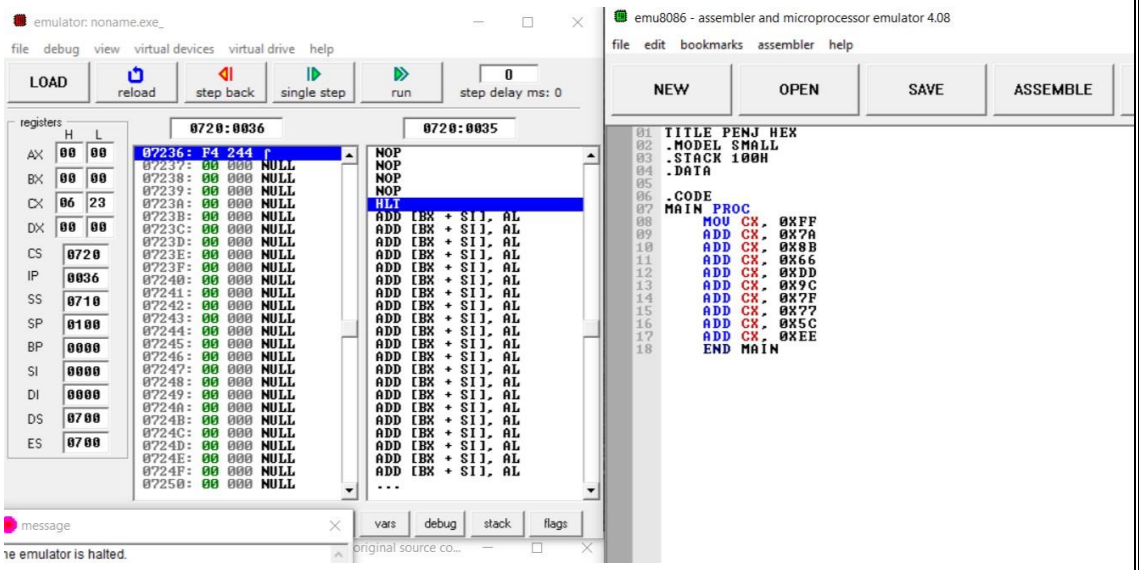


   b) Perintah Loop

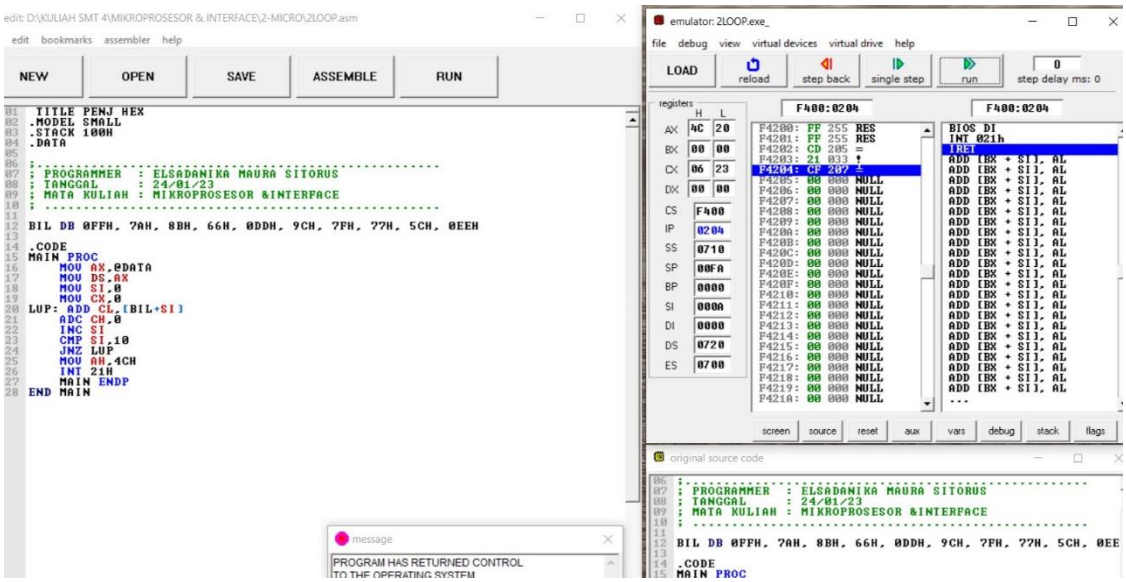


2. FFH, 7AH, 8BH, 66H, DDH, 9CH, 7FH, 77H, 5CH, dan EEH. Simpan di CX.

a) Perintah 16 bit

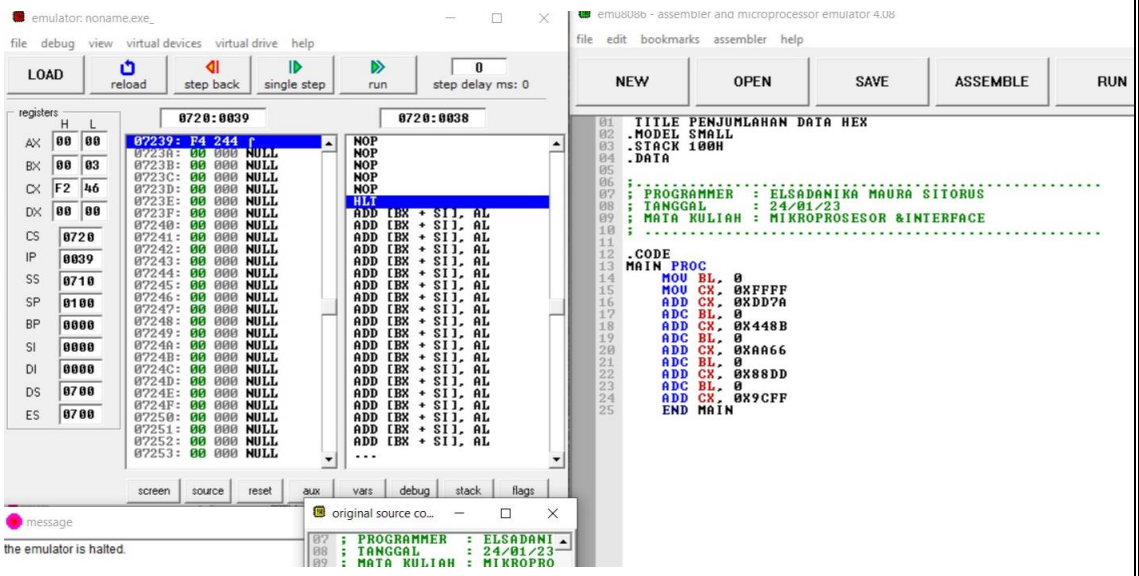


b) Perintah Loop

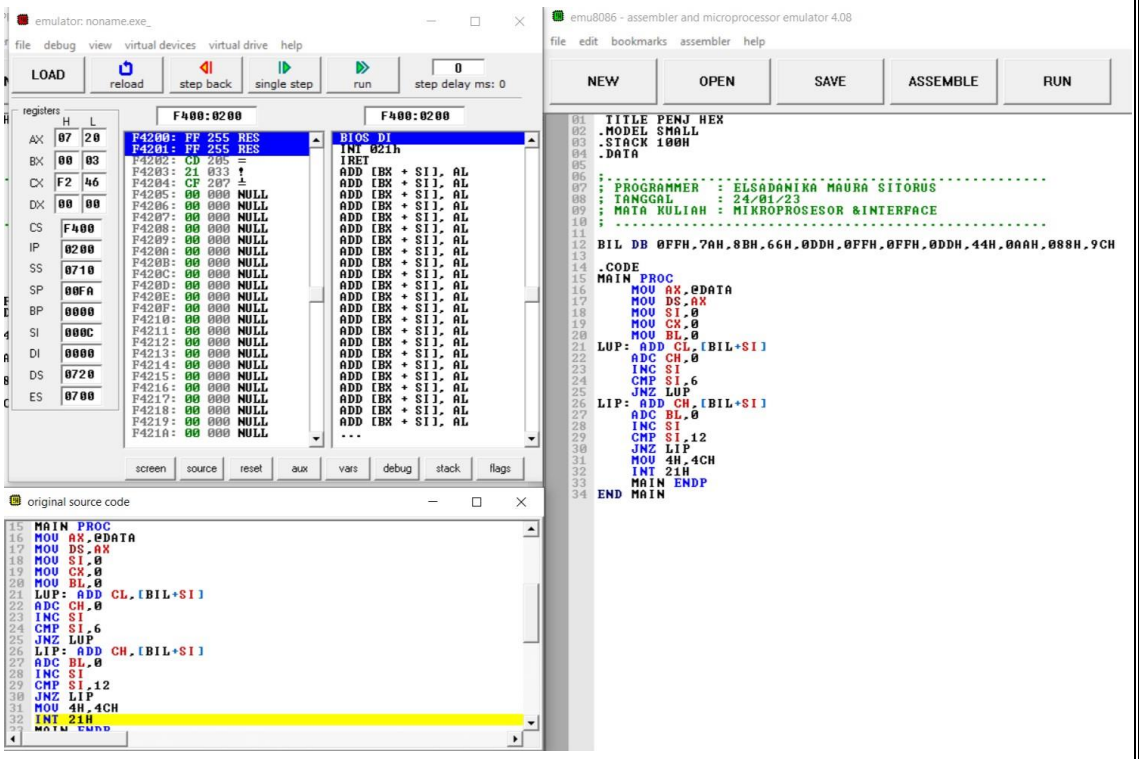


3. FFFFH, DD7AH, 448BH, AA66H, 88DDH, 9CFFH. Simpan di BL:CX.

a) Perintah 16 bit

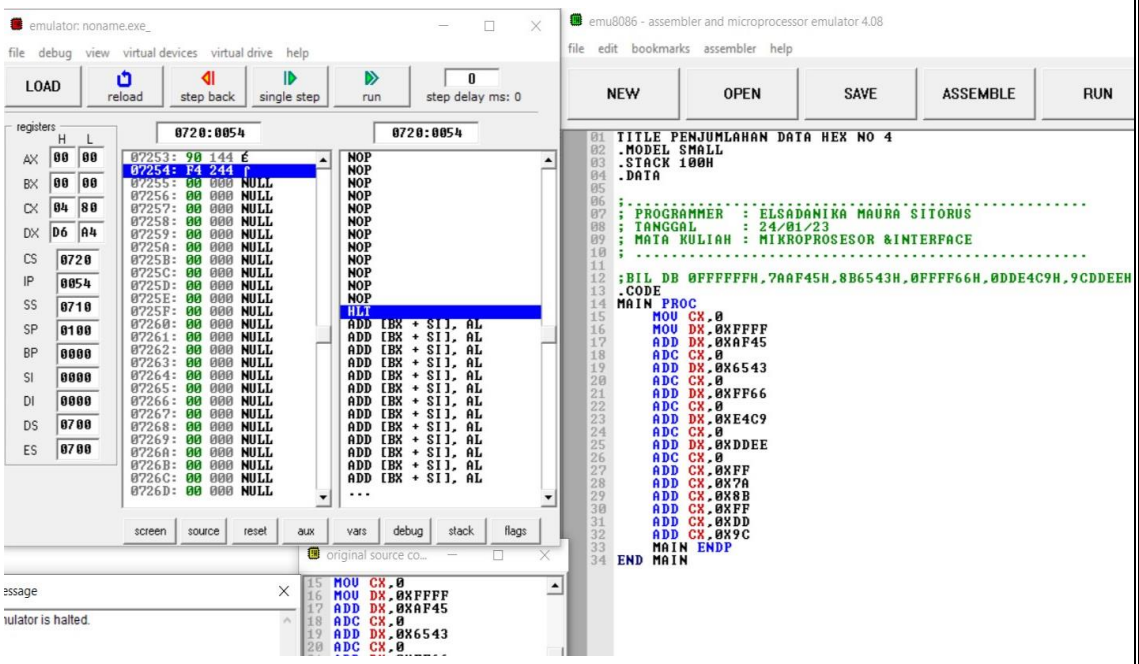


b) Perintah Loop

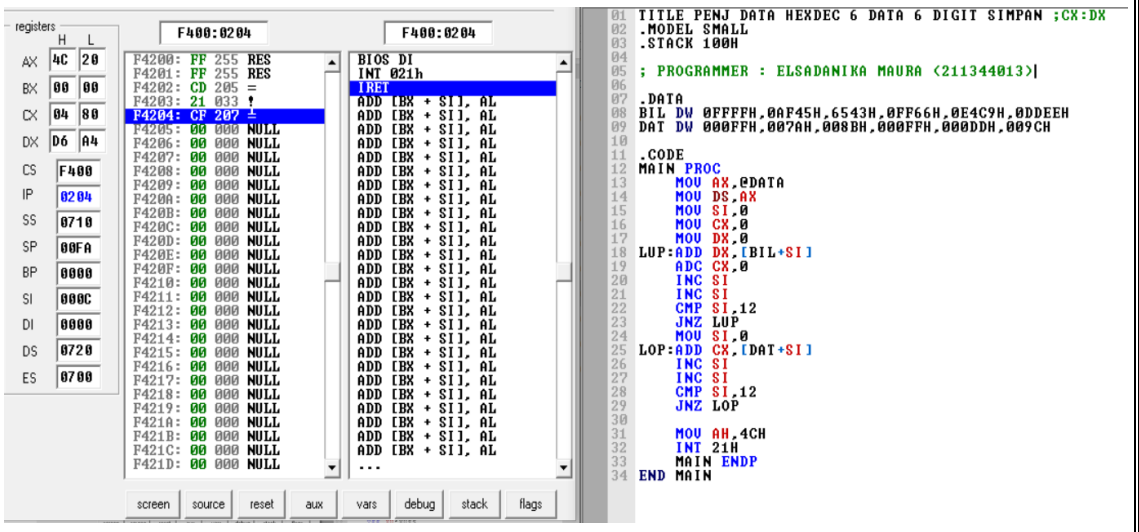


4. FFFFFFH, 7AAF45H, 8B6543H, FFFF66H, DDE4C9H, 9CDDEEH. Simpan di CX:DX.

   a) Perintah 16 bit

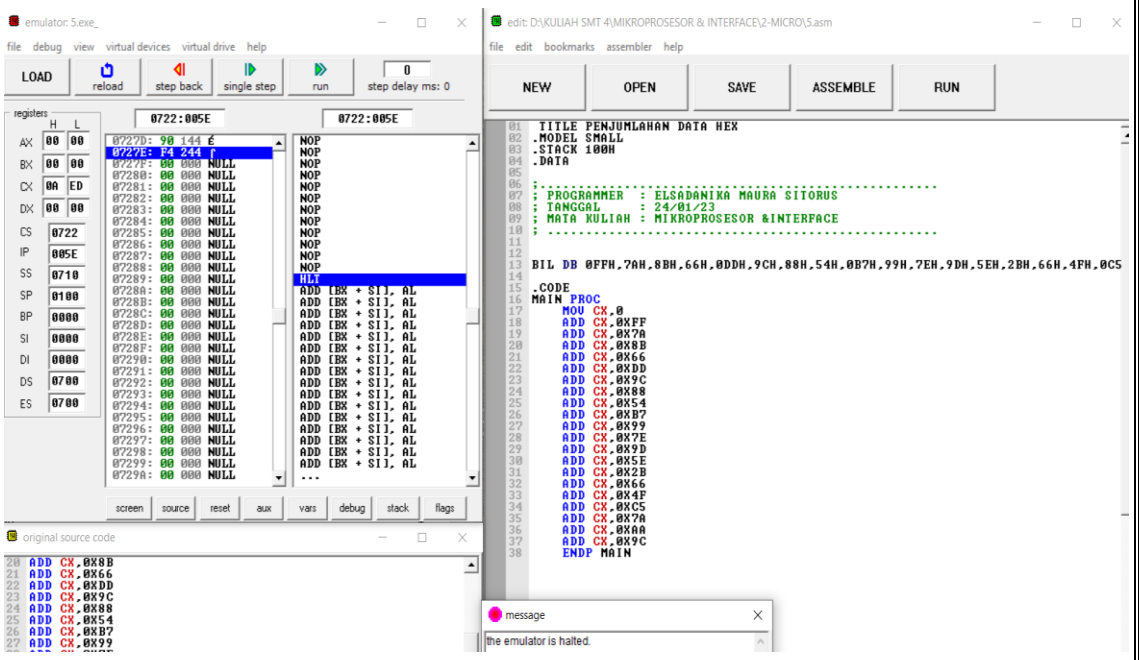


   b) Perintah Loop

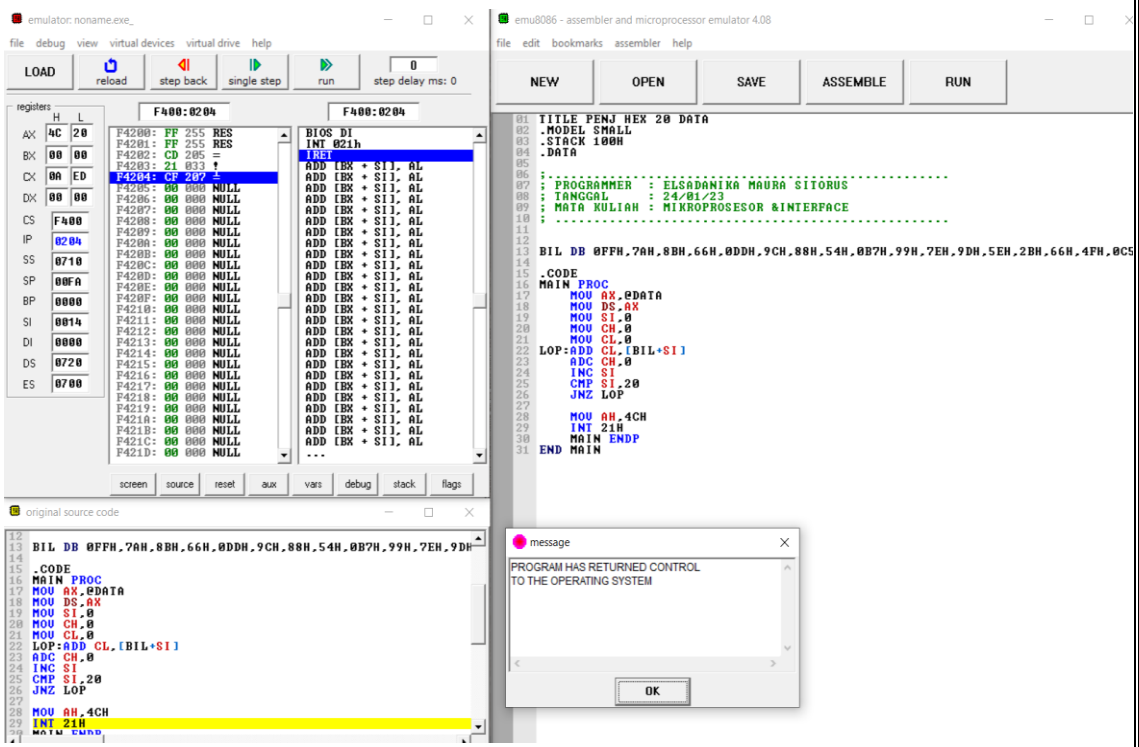


5. FFH, 7AH, 8BH, 66H, DDH, 9CH, 88H, 54H, B7H, 99H, 7EH, 9DH, 5EH, 2BH, 66H, 4FH, C5H, 7AH, AAH, 9CH. Simpan di CX.

   a) Perintah 16 bit



   b) Perintah Loop

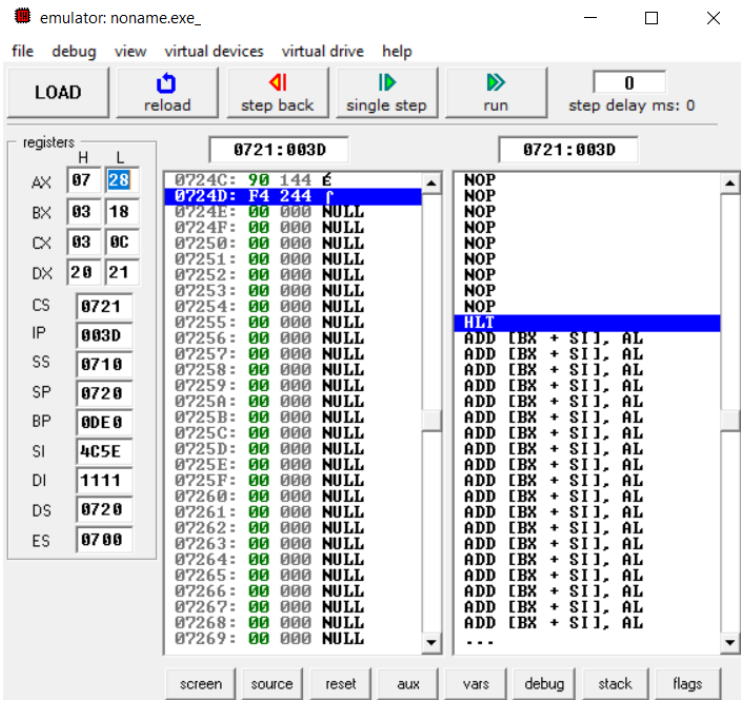


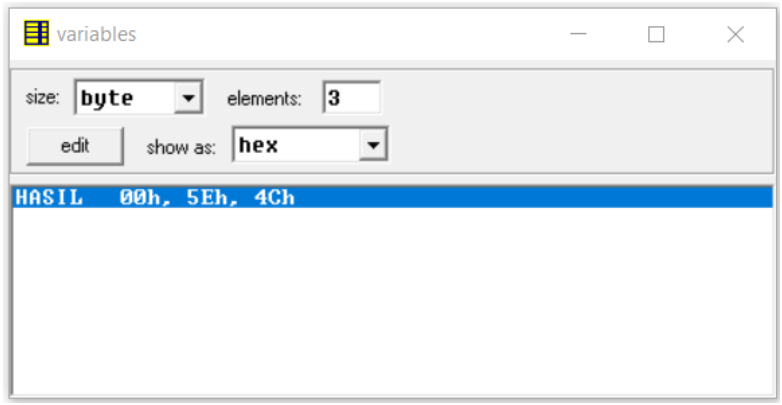
6. AX, BX, CX, DX, BP, DI. Simpan di data area HASIL.

```
TITLE PENJ DARI REG 0728+0318+030C+2021+0DE0+1111=4C5E
.MODEL SMALL
.STACK 100H

; PROGRAMMER : ELSADANIKA MAURA SITORUS

.DATA
HASIL DB 0
      DW 0
.CODE
MAIN PROC
    MOV SP,@DATA
    MOV DS,SP
    MOV SI,0
    ADD SI,AX
    MOV AL,0
    ADC AL,0
    ADD SI,BX
    ADC AL,0
    ADD SI,CX
    ADC AL,0
    ADD SI,DX
    ADC AL,0
    ADD SI,BP
    ADC AL,0
    ADD SI,DI
    ADC AL,0
    MOV [HASIL],AL
    MOV WORD PTR[HASIL+1],SI
    MAIN ENDP
END MAIN
```





F. Analisis Data

Praktikum penjumlahan data-data heksadesimal pada intel 8086 menggunakan simulator EMU8086 yang telah dilakukan menghasilkan hasil data yang sesuai antara perhitungan teori atau manual dengan perhitungan berdasarkan program yang telah dibuat. Berikut ini adalah analisis data setiap program yang dibuat yang dibagi dalam urutan nomor soal:

1. FFH, 7AH, 8BH, 66H, DDH, 9CH. Simpan di CX.

   Menurut perhitungan teori/manual, didapatkan bahwa hasil penjumlahan data tersebut adalah 03E3. Untuk menjumlahkan data secara perintah 16 bit, maka diperlukan instruksi MOV CX,0 untuk menyatakan bahwa register cx adalah 0 dan memastikan belum ada data yang di-copy ke register CX. Selanjutnya, diperlukan perintah ADD untuk menjumlahkan data satu persatu tanpa perlu menambahkan instruksi ADC karena data dijumlahkan pada register CX yang meliputi CH dan CL serta data bernilai 8 bit sehingga carry terjadi dari CL ke CH yang sudah cukup diwakilkan oleh CX.

   Untuk menjumlahkan data dengan perintah loop, maka diperlukan deklarasi memory area dengan tipe DB dikarenakan data-data yang akan dijumlahkan adalah bernilai 8 bit/1 byte. Setelah itu, pada bagian kode program dilakukan setting awal bahwa data-data yang ada pada memory area akan dialamatkan ke register DS dan AX. Hal tersebut dinyatakan dengan MOV AX,@DATA dan MOV DS,AX. Selain itu diperlukan juga instruksi Source Index atau SI untuk memberikan penomoran pada data di memory area. Diberikan label LUP: dengan perintah-perintah dalam loop adalah seperti yang dilampirkan pada data hasil praktikum. Perumusan yang diberikan pada perintah ADD di dalam loop disesuaikan dengan memory area, pada program pertama perumusan yang digunakan adalah [BIL+SI].

   Perintah INC SI akan menaikkan perhitungan secara SI+1 lalu CMP SI,6 akan membandingkan nilai SI. Ketika SI sudah mencapai data ke 7 maka proses looping akan berhenti dan melanjutkan ke perintah program selanjutnya yaitu stop program. Perintah JNZ LUP digunakan untuk menjalankan loop dengan kondisi jump if not zero yang menyesuaikan dengan CMP SI. Ketika SI < 7 maka akan bernilai logic 1 dan saat mencapai SI=7 maka bernilai logic 0 dan program melanjutkan ke perintah

setelah loop. Dengan demikian didapatkan hasil output program pada register CX bernilai 03E3 sesuai dengan perhitungan teori.

2. FFH, 7AH, 8BH, 66H, DDH, 9CH, 7FH, 77H, 5CH, dan EEH. Simpan di CX.

   Program kedua memiliki kesamaan tipe dan jenis dengan program pertama. Perbedaan yang terdapat hanyalah jumlah data yaitu menjadi 10 data. Dengan kesamaan ini maka program untuk menjumlahkan data-data pada perintah 16 bit perlu ditambah dengan penjumlahan data baru. Sedangkan untuk perintah loop, data-data baru perlu dimasukkan ke memory area serta dilakukan perubahan batas compare pada CMP SI menjadi 10. Dengan demikian, didapatkan hasil yang bersesuaian dengan perhitungan manual, yaitu 0623 yang ditempatkan pada register CX.

3. FFFFH, DD7AH, 448BH, AA66H, 88DDH, 9CFFH. Simpan di BL:CX.

   Untuk perintah 16 bit, maka diberikan instruksi awal yaitu MOV BL,0 untuk memastikan bahwa register BL masih bernilai 0. Setelah itu perintah MOV CX dan ADD CX dengan diikuti ADC BL,0 untuk setiap setelah ditambahkan dengan data baru. Dengan demikian didapatkan hasil bernilai 03F246 yang bertempat di register BL:CH:CL.

   Untuk perintah penjumlahan dengan menggunakan loop, digunakan deklarasi memory area dengan tipe DB dikarenakan data-data yang akan dijumlahkan diubah menjadi bernilai 8 bit/1 byte dengan membagi data berdasarkan low byte dan high byte nya. Pada memory area, diurutkan data mulai dari low byte, lalu dilanjutkan dengan data-data high byte. Digunakan perintah loop sebanyak dua kali. Loop pertama digunakan untuk menjumlahkan data-data low byte dan hasil outputnya disimpan pada register CL dengan carry pada register CH. Digunakan increment sebanyak 1 kali dengan compare CMP SI,6 untuk menjumlahkan data ke 0-5 yang bernilai low byte. Jika loop pertama telah selesai, maka dilanjutkan ke loop kedua.

   Loop kedua digunakan untuk menjumlahkan data-data high byte. Output dari penjumlahan ini disimpan pada register CH dengan carry yang disimpan di register BL. Increment dilakukan sebanyak 1 kali dengan compare CMP SI,12 untuk menjumlahkan semua data dengan carry. Ketika telah dicapai data ke 12 maka akan bernilai logic 0 sehingga dilanjutkan dengan instruksi stop program. Dengan

demikian didapatkan hasil output yang bersesuaian antara teori, perintah 16 bit, dan perintah loop yaitu 03F246 yang disimpan pada register BL:CH:CL.

4. FFFFFFH, 7AAF45H, 8B6543H, FFFF66H, DDE4C9H, 9CDDEEH. Simpan di CX:DX.

   Dikarenakan pada 6 data ini masing-masing bernilai 6 digit, maka perlu dibagi 2 menjadi 16 bit dan 8 bit. Data diurutkan dari setiap data low byte yaitu 4 digit paling belakang tiap data, lalu diikuti dengan 2 digit awal setiap data yaitu high byte. Untuk perintah penjumlahan 16 bit maka pada badan program diawali dengan perintah MOV CX,0 dan MOV DX,0XFFFF selanjutnya dilanjutkan dengan ADD pada register DX dengan ADC yang disimpan pada register CX. Dilakukan penjumlahan untuk low byte terlebih dahulu dan dilanjutkan dengan penjumlahan high byte. Pada bagian ADD penjumlahan high byte yang hanya mengandung 8 bit tiap datanya, tidak perlu digunakan carry ADC, karena data-data high byte dijumlahkan langsung ke register CX. Tidak lagi menjumlahkan di register DX.

   Untuk perintah penjumlahan dengan looping, dibutuhkan dua deklarasi data dengan 2 variabel yang berbeda untuk menempatkan data 16 bit low byte dan 8 bit high byte (yang ditulis dalam bentuk 16 bit dengan penambahan 00 pada awal tiap data) yang telah dibagi sebelumnya. Data low byte ditempatkan pada variabel BIL DW dan data high byte ditempatkan pada variabel DAT DW. Digunakan jenis DW untuk data-data ini baik pada low byte data type maupun high byte data, sehingga pada proses loop increment akan dilakukan sebanyak dua kali. Pada program loop pertama hasil penjumlahan disimpan pada register DX dengan perumusan [BIL+SI] digunakan juga instruksi ADC CX,0 untuk menyimpan hasil carry ke register CX. Setelah itu digunakan instruksi increment INC SI sebanyak dua kali dan dilanjutkan dengan compare CMP SI,12. Hal ini dikarenakan data bernilai 16 bit yang berarti akan mengisi memory area sebanyak jumlah data dikali 2. Jika telah tercapai perbandingan hingga SI=12 maka dilanjutkan dengan JNZ loop pertama dan SI perlu kembali diisi dengan 0 maka diinstruksikan MOV SI,0 pada akhir loop pertama.

   Loop kedua hasil penjumlahan disimpan ke register CX dengan perumusan [DAT+SI] sehingga yang akan dijumlahkan adalah data-data pada variabel DAT DW yang merupakan data high byte. Dilanjutkan dengan instruksi INC SI sebanyak

2 kali karena tipe variabel pada memory area yang dipilih adalah DW. Perintah CMP SI,12 digunakan pula dikarenakan data high byte dibentuk dalam 16 bit. Diakhiri dengan instruksi JNZ loop kedua. Sehingga dengan demikian data-data akan dijumlahkan dengan sesuai yaitu low byte terlebih dahulu yang diikuti penjumlahan high byte. Didapaatkan hasil yang bersesuaian antara perhitungan manual, perintah 16 bit, perintah loop yaitu 0480D6A4 yang disimpan pada register CX:DX.

5. FFH, 7AH, 8BH, 66H, DDH, 9CH, 88H, 54H, B7H, 99H, 7EH, 9DH, 5EH, 2BH, 66H, 4FH, C5H, 7AH, AAH, 9CH. Simpan di CX.

   Program kelima memiliki kesamaan tipe dengan program pertama dan kedua. Perbedaan yang terdapat adalah jumlah data yaitu 20 data 8 bit berbeda. Dengan kesamaan ini maka program untuk menjumlahkan data-data pada perintah 16 bit perlu ditambah dengan penjumlahan data-data sehingga akan terdapat instruksi ADD sebanyak 20 buah. Sedangkan untuk perintah loop, 20 data perlu dimasukkan ke memory area serta dilakukan perubahan batas compare pada CMP SI menjadi 20. Dengan demikian, didapatkan hasil yang bersesuaian antara perhitungan manual, perintah 16 bit, dan perintah loop yaitu 0AED yang ditempatkan pada register CX.

6. AX, BX, CX, DX, BP, DI. Simpan di data area HASIL.

   Program keenam ini mengisikan data heksadesimal langsung pada register ketiga program dijalankan. Untuk dapat bekerja seperti yang diinginkan, maka pada bagian data dideklarasikan variabel HASIL DB 0 dan DW 0. Hal ini dilakukan untuk membuat data pada memory area menjadi 0. Pada badan program, digunakan instruksi awal yaitu MOV SP,@DATA supaya semua data yang nanti diisikan masuk ke register SP, dilanjutkan dengan instruksi MOV DS,SP dan MOV SI,0 sehingga data SI dimulai dari 0. Penjumlahan dilakukan pada register SI sedangkan carry dimasukkan ke register AL. Program diakhiri dengan instruksi MOV [HASIL],AL dan MOV WORD PTR [HASIL+1],SI sehingga data hasil penjumlahan yang ada di SI dan data carry di AL masuk ke data HASIL.

   Untuk menampilkan hasil data area HASIL, maka dapat dilihat pada bagian vars ketika program telah dijalankan dan telah diisikan data-data yang ingin dijumlahkan. Pada program keenam, programmer menjumlahkan data 0728H + 0318H + 030CH + 2021H + 0DE0H + 1111H = 4C5E. Hasil yang ditampilkan pada vars HASIL pun sesuai dengan perhitungan manual yang telah dilakukan.

G. Kesimpulan

Melalui praktikum yang telah dilakukan, diketahui bahwa terdapat cara yang lebih efektif untuk menjumlahkan bilangan heksadesimal melalui program loop. Pada program dengan loop, perintah yang digunakan menjadi lebih sedikit dan tidak perlu menjumlahkan tiap data secara satu-persatu. Parameter-parameter yang penting dalam membuat program dengan loop MOV AX,@DATA ; MOV DS,AX ; MOV SI ; label ; INC SI ; CMP SI ; JNZ label.

Untuk data bernilai 8 bit maka digunakan Define Byte atau DB, sedangkan data bernilai 16 bit atau 4 digit angka dan/atau huruf heksadesimal maka digunakan DW atau Define Word. Setiap increment INC SI disesuaikan dengan penggunaan DW atau DB, INC SI dilakukan sebanyak 1 kali jika tipe data adalah DB dan dilakukan sebanyak 2 kali jika tipe data adalah DW. Data-data yang disimpan pada memory area selalu bernilai 8 bit, sehingga ketika terdapat data yang bernilai 16 bit akan terbagi menjadi 2 data pada memory area. Selain itu diketahui bahwa setiap data yang diawali dengan huruf maka perlu ditambahkan 0 di awal data supaya dapat dideteksi. Register DS harus diisi dengan alamat segment, selain itu penulisan label untuk loop harus tidak memiliki spasi dengan tanda titik duanya.

H. Daftar Pustaka

[1] Kho, D. Pengertian Mikroprosesor (Microprocessor) dan Cara Kerja Mikroprossesor. [teknikelektronika.com]. Available: https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerja-mikroprosesor-adalah/

[2] Hidayatullah, S. 2021. Pengertian Mikroprosesor Beserta Fungsi dan Sejarah Mikroprosesor. [belajaronline.net]. Available: https://www.belajaronline.net/2021/01/pengertian-mikroprosesor-fungsi-dan-sejarah-mikroprosesor.html

[3] Ardiyanto88. 2013. Perintah Dasar Bahasa Assembly Menggunakan Emulator 8086. [ardiyanto88.wordpress.com]. Available: https://ardiyanto88.wordpress.com/2013/10/25/perintah-dasar-bahasa-assembly-menggunakan-emulator-8086/

I. Lampiran ASCII Code Table

ASCII Table

The ASCII code is used to give to each symbol / key from the keyboard a unique number called ASCII code.
It can be used to convert text into ASCII code and then into binary code. It can be used within your code to identify specific characters in a string or specific keys being pressed on the keyboard.

The ASCII table contains 256 codes (from 0 to 255). The table below only shows the most useful ASCII codes.

Char	ASCII Code (Decimal)
a	97
b	98
c	99
d	100
e	101
f	102
g	103
h	104
i	105
j	106
k	107
l	108
m	109
n	110
o	111
p	112
q	113
r	114
s	115
t	116
u	117
v	118
w	119
x	120
y	121
z	122

Char	ASCII Code (Decimal)
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Char	ASCII Code (Decimal)
space	32
!	33
"	34
#	35
$	36
%	37
&	38
'	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47
:	58
;	59
<	60
=	61
>	62
?	63
@	64
[	91
\	92
]	93
^	94
_	95
`	96
{	123
|	124
}	125
~	126
‘	145
’	146
“	147
”	148
•	149
˜	152

Char	ASCII Code (Decimal)
0	48
1	49
2	50
3	51
4	52
5	53
6	54
7	55
8	56
9	57

Char	ASCII Code (Decimal)
€	128
£	163
¥	165
$	36
©	169
™	153
°	176
˜	152
¡	161
¿	191

© 101 Computing – www.101Computing.net